The paper, in plain English

Deterministic Guardrails and Variance-Aware Evaluation
for Multi-Agent Content Personalization

Prachi Sharma Danish Kaul Dishant Sherawat

School of Engineering and Technology, Sushant University, Gurugram, Haryana

A companion to the research paper. No jargon that is not explained.

1. Is the paper about our project?

Yes, entirely. It is not a literature survey or a side essay. Every number in it was produced by running our own code, and every section describes something we actually built.

Paper section	Code that implements it	Size
3.1 Eight-stage pipeline	`app/agents/graph.py`	461 lines
3.2 Three-level profiles	`app/services/personalization.py`	466 lines
3.3 Scope guard	`app/agents/scope.py`	138 lines
3.4 Grounding guard	`app/agents/grounding.py`	178 lines
4.1 The test dataset	`marketing_eval/cases.toml`	30 cases
4.2 Automatic checks	`marketing_eval/harness.py`	110 assertions
4.3 Rubric and judge	`rubric.md`, `judge.py`	32 dimensions
5.3 Model comparison	`compare_models.py`	7 runs
5.6 The empty-reply defect	`llm/openai_compatible.py`	in `generate()`

The system is 76 Python modules, roughly 12,600 lines of application code, and 10 verification scripts. One of those scripts, `check_claims.py`, re-reads the stored run reports and confirms that all 76 numbers in the paper still match them. So if anyone asks where a figure such as 4.28 comes from, the answer is a file on disk, not somebody's memory.

2. What problem were we solving?

Ask a language model to write marketing copy and two things go wrong.

It does not know your brand. It writes in a generic voice, picks the wrong conventions for the platform, and ignores the fact that you never use emojis.

It invents numbers. Ask for a post about cloud costs and it will cheerfully write "cut your bill by 40%" — a figure nobody gave it. In marketing copy that is not a harmless slip. It is a false claim published under your brand name.

> We tried the obvious fix for the second problem first: we told the model, in its instructions, not to invent figures. **It kept inventing figures.** That failure shaped the entire project.

3. What we built

Five narrow agents rather than one general chatbot: a caption writer, a search-optimization agent, a campaign planner, and two that produce images (a social post image and an advertisement creative). Each has its own instructions and its own settings.

Three parts are worth understanding properly.

3.1 Three-level profiles

Your details live at the level where they are actually true.

- **Global** — who you are, how you like to be spoken to.
- **Domain** (marketing) — brand name, industry, audience, tone, banned words.
- **Agent** — for the caption writer: which platforms, how long, hashtag policy.

State your brand tone once and all five agents receive it. State "at most three hashtags" and only the caption writer receives it, because it is meaningless to the others.

3.2 The scope guard

If you are in the caption writer and you ask for an advertisement image, the system **hands you off** to the right agent instead of doing a poor job.

The important part: that decision is made in **plain code, with no model call.** It is instant and free. It compares how much evidence your request gives for the current agent's job against every other agent's job, and redirects only when the mismatch is obvious — four separate conditions must all hold, because wrongly redirecting somebody is worse than answering slightly off-topic.

> **A bug worth mentioning to an examiner.** The first version scored each agent using its whole vocabulary *including its own name*. So "design an ad creative" sent to the caption writer scored highly *for the caption writer* — because the phrase contains words from its own name — and was never redirected. The fix was to score only *task* words and ignore name matches.

3.3 The grounding guard

After the model writes something, we scan the text for claim-shaped numbers — percentages, money, "10x", "up to" — and check each one against the numbers you actually supplied in your request, your attachment or your profile. Anything we cannot account for is **flagged**.

Two design decisions matter here.

It flags; it never rewrites. Deleting a number mid-sentence leaves you with "reduce costs by up to", which is worse than the original. And only *you* know whether the claim is true, so the system's job is to tell you, not to decide for you.

It checks provenance, not truth. We are not fact-checking the world. We are asking a much cheaper and more useful question: did this number come from the user, or did the model make it up?

4. How we measured it, and why that is the real contribution

This is the part a reviewer cares about most.

4.1 Two kinds of question

Some questions a computer can answer exactly:

- Did it hand off to the right agent?
- Did the budget split add up to the 400 the user gave?
- Did it stay under three hashtags?
- Is the generated image exactly 1200×627 pixels?

We wrote **110 such checks** across 30 test cases. They run in seconds, cost nothing, and give the same answer every time.

Other questions a computer cannot answer: is this a good opening line? Does it sound like our brand? Would you actually publish it? For those we used an **LLM judge** — a second, independent AI model

that reads the output against a written scoring guide (32 dimensions, scored 1–5) and returns a score with a reason.

4.2 Then we checked whether the judge could be trusted

Most people stop at "the judge gave us 4.23 out of 5" and report it. We did not, and this is where the interesting results came from.

We asked the judge the same question three times — same outputs, same prompt, temperature set to zero, so it should be deterministic. It disagreed with itself, by a mean spread of **0.51 to 0.54 points on a five-point scale**.

Then we re-ran the whole system without changing anything and judged again. One agent's score moved by **0.37** with zero changes to the code.

> That single number reframed everything. It means any quality difference smaller than about half a point is **noise, not a result.**

5. What we found

5.1 Finding 1: the guards worked

Holding the model fixed, adding the guards took the automatic test suite from **102/105 to 105/108** passing checks. Three failures remained, and no amount of prompt rewriting fixed them.

We then swapped in a bigger model and changed nothing else. All three passed and the suite reached **110/110**. So those three failures were the *model's* limitation, not our design's. You can only make that claim if you change one thing at a time, which is what a controlled experiment is for.

5.2 Finding 2: our scoring scale had silently broken

The rubric defines what a 1, a 3 and a 5 look like for each dimension. We assumed the judge would use 2 and 4 for work that fell in between.

It never did. Across **174 scores, every single one was a 1, a 3 or a 5.** Not one 2, not one 4. Our five-point scale had quietly become a three-point scale, and nothing in the system would ever have told us — we saw it only because we plotted the distribution.

Naming the middle values in the judge's instructions brought them back and cut the worst disagreement from 4 points to 2.

5.3 Finding 3: "best model for each agent" sounds clever and is not

We had five specialised agents, so the natural next step was to give each one the model best suited to it. A first screening pass appeared to confirm it: two agents looked 0.45 and 0.33 better under different models.

So we built it properly and tested it against a control.

	One model for all	Mixed fleet
Overall score	**4.28**	4.18
Speed	**54 s**	201 s
The two "improved" agents	4.56 / 3.80	4.33 / **3.87**

The mixed fleet was **worse overall and 3.7 times slower**, and one of the two "improved" agents actually got *worse*. A proper statistical test (Wilcoxon signed-rank, 15 changed pairs) gave $p = 0.42$, nowhere near significant.

Why did screening mislead us? Because 0.33 and 0.45 sit *inside* the noise floor we had measured. They were never real.

> We report this as a failure, in full, in the paper. That is deliberate. A negative result that saves somebody else three days of work is worth more than a positive result we had to squint at.

5.4 Finding 4: a bug that looked like somebody else's fault

While comparing models, four of them failed on between 4 and 19 test cases each, reporting what looked like the provider being down.

It was not. Some newer models "think" before answering, and that thinking consumes the token budget. When it runs out mid-thought, the interface returns **success, with an empty message**. Our code saw an empty message and reported a provider outage. One model returned 700 tokens of pure thinking and *zero* characters of answer, on one request in six.

We now detect that exact signature and retry with more room. Five of six models went to **zero errors**. The sixth is genuinely unusable for this task: it produced 12,000 to 16,000 characters of reasoning and never answered, needing 41 retries in a single run.

This mattered beyond robustness. Before the fix, each model failed on a *different* set of cases, so each model's score described a different subset of the test. Any ranking would have been an accident of which cases happened to survive.

6. The one-sentence version of each takeaway

1. **If you can check something in code, check it in code.** Telling a model "do not invent numbers" does not work. A 178-line checker does.
2. **Flag, do not fix.** When the system cannot know whether something is wrong, tell the user instead of silently editing.
3. **Publish your noise floor, not just your score.** We measured ours at half a point, and it invalidated two conclusions we had already drawn.
4. **Specialised agents do not need specialised models.** We tested it; the difference was smaller than the noise and cost 3.7 times the latency.
5. **Test your test code.** Both of our worst bugs were in the *measuring* equipment, and both produced confident, wrong numbers.

7. Honest limitations

Expect to be asked about these. They are stated plainly in the paper's Threats to Validity section, which is the correct place for them.

- **No human scoring pass.** Our judge was never checked against real human raters. This is the biggest weakness, and we say so.
- **One brand, one language, one industry.** All results use a single synthetic brand in English.
- **We wrote the 30 test cases ourselves**, to probe failures we expected. They are not sampled from real traffic.
- **Small sample for the statistics.** Fifteen changed pairs. A larger dataset might find an effect we could not.
- **The noise floor of 0.5 is ours**, for this task and this judge. It is not a universal constant.

8. Likely questions, and how to answer them

Why not just fine-tune a model on the brand?
Too expensive per user, and it cannot be updated the moment a brand changes its tone. Composing the profile at request time is instant and works per user.

Is using an AI to judge an AI not circular?
Yes, which is why we measured how much it disagrees with itself, used a *different* model as judge than the one being judged, and made the code flag loudly whenever those two are the same. Section 2.4 of the paper cites the published work on self-preference bias.

Why is a negative result in the paper?
Because it is true, and because the reasoning behind it — effect size against measured noise — is the paper's main methodological point. Removing it would leave out the most useful finding.

What is actually new here?
Three things: scope enforcement as a deterministic, zero-cost step rather than a prompt instruction; provenance-checking of claim-shaped numbers rather than fact-checking; and reporting judge variance as a required part of judge-based evaluation.

Could you have used a bigger model and skipped all this?
A bigger model fixed three test failures, and we show that. It did *not* stop figure fabrication, and it cannot perform the scope handoff for free. The guards and the model solve different problems.